

CLEAN in R: A Practical Tutorial

Lingyu (Jack) Fuca

Overview

This tutorial shows how to apply the **CLEAN** package to network data for detecting and reconstructing latent homophilous information. **CLEAN** takes a sequence of adjacency matrices and returns recovered homophily based on the selected model, assumptions, and setups. By placing the recovered homophily back into the original structural model, the confounding effect between contagion and homophily in the same social network is de-biased.

Installation

You may directly run this chunk to install the package.

```
install.packages("remotes")
remotes::install_github("fucailanju/CLEAN")
```

Please note that right now (late January, 2026) we only provide stochastic-block-model-based latent homophily detection and reconstruction. Note that SBM-based homophily points to discrete memberships and clear boundaries between each cluster. Usually, it indicates the probability of an edge to be formed between any two nodes aligns with this principle: $p_{\text{within}} \gg p_{\text{between}}$. For more information on SBM, please check the `sbm` package in R and these most important papers:

- *Stochastic block models: First steps* by Paul W. Holland, Kathryn Blackmond Laskey, Samuel Leinhardt.
- *Stochastic block models and community structure in networks* by Brian Karrer, M. E. J. Newman.
- *Community Detection and Stochastic Block Models: Recent Developments* by Emmanuel Abbe.

Latent-space-model-based latent homophily detection and reconstruction will be added to **CLEAN** in a few weeks. Note that LSM focuses on continuous Euclidean distance between any two nodes to predict the probability of edge formation. This difference compared to SBM indicates that homophily projects to different substantive features in the real world, such as ideology, real geographic distance, membership of IOs, and so on.

CLEAN can be applied to adjacency matrices either varying by time or static. Depending on the settings, only for adjacency matrices varying by time will the recovered homophily be plugged on both the temporal and membership (SBM) / dimension (LSM) tags. For example, if an SBM-stemmed adjacency matrix does **not** involve time, the outcome is expected to be like `comm_1`, `comm_2`, etc. Its LSM counterpart should generate outcomes such as `xhat_1`, `xhat_2`, etc, which are essentially proxies of the true yet latent continuous dimensions. In **CLEAN** package, we develop `clean_blocks()` function for this task.

While for time-varying SBM-stemmed homophily, for example, if the time-interval is yearly, the outcomes are expected to be like `comm_1_1999`, `comm_2_2005`, etc. In **CLEAN** package, we develop `clean_blocks_by_time()` function for this task. Again, the LSM counterpart will be developed in the next version of the package.

Input data format

Users must prepare the network adjacency information **before** calling **CLEAN**. **CLEAN** accepts either:

- a **single adjacency matrix** (temporally-static networks), or
- a **named list of adjacency matrices** indexed by time (temporally-varying networks).

(1) Temporally-static network data

Provide one $n \times n$ adjacency matrix **A**:

- $A[i, j]$ encodes the tie from node *i* to node *j*
- **A** must be square (`nrow(A) == ncol(A)`) and contain **no NA**
- If **A** has `rownames()`/`colnames()`, they should be identical and represent node IDs

Recommended (safe) format: include node IDs as row/column names so **CLEAN** can align nodes correctly.

(2) Temporally-varying network data

Provide a **named list** of adjacency matrices, `A_by_time`, where each element corresponds to one time period:

- `A_by_time` must be a `list`
- `names(A_by_time)` must exist and match the time values in the panel data:
`names(A_by_time) == unique(as.character(df[[time]]))`
- Each `A_by_time[[t]]` must be square and contain **no NA**

Node alignment rule (important): For each time period `t`, `CLEAN` aligns the adjacency matrix to the IDs that appear in the panel data at that time. The most robust way to ensure correct alignment is:

- set `rownames(A_by_time[[t]])` and `colnames(A_by_time[[t]])` to node IDs
- ensure `rownames(A_by_time[[t]]) == colnames(A_by_time[[t]])`

`CLEAN` will then subset/reorder the matrix to match the panel IDs for time `t`.

Example (time-varying):

```
A_by_time <- list(  
  "2000" = A2000,  
  "2001" = A2001  
)
```

Exemplified application

We use Olar's 2019 paper *Do they know something we don't? Diffusion of repression in authoritarian regimes* to show the entire operational process. In this paper, there are two time-varying adjacency matrices, one based on institutional similarity and the other based on experiential similarity. Due to how these two adjacency matrices are conceptualized, measured, and built, we believe that SBM is the appropriate model because political institutions and experience of civil-war casualties are memberships and/or boundaries, but not something that is continuous.

```
library(haven)  
library(dplyr)  
library(ggplot2)  
library(CLEAN)  
  
# Set up your own working directory  
setwd()
```

```
# The original dataset for the original structural model
data <- read_dta("diffusion_repression.dta")

# One of the two time-varying adjacency matrices as the example
coop <- read_dta("W_coop.dta")

strctr <- read_dta("W_structural.dta")
```

To prepare the proper inputs that **CLEAN** expects, some pre-analysis are necessary before you work with **CLEAN**.

```
# Check the dimensions of one of the adjacency matrices
rs <- rowSums(as.matrix(coop)) # 4171 in total
cs <- colSums(as.matrix(coop)) # 4171 in total

# Check the number of times across all over from the original
# dataset, the unit of time is `year`
each_yr <- table(data$year)
length(each_yr) # [1] 58, which means there are 58 years in total

# Split each year from each other
# Still inside the original dataset, now checking **cumulative** numbers
# of how many observations are there as years accumulates, so that we can
# later decide how to split the integrated adjacency matrix into
# yearly-based adjacency matrices
yr_fin <- cumsum(each_yr)
print(yr_fin)
```

```
> print(yr_fin)
1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965
   39   79  120  163  207  251  294  340  384  430  477  539  604  673  744
1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980
  817  893  972 1056 1141 1228 1315 1405 1495 1586 1678 1774 1870 1965 2058
1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995
2151 2244 2336 2427 2517 2606 2693 2779 2863 2943 3016 3088 3165 3238 3307
1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008
3376 3447 3520 3592 3663 3728 3792 3857 3923 3988 4050 4111 4171
```

This table reads as, till the end of which year, how many observations are there from No.1 to the last observation of that year. For example, the year 1960 points to 430. It means, till the end of year 1960, there are in total 430 observations accumulated from the end of 1951.

```
# Calculate the starting index for each year by subtracting
# the number of observations per year from yr_fin and adding 1.
yr_start <- yr_fin - each_yr + 1

print(yr_start)
```

Here are the results:

```
> print(yr_start)

1951 1952 1953 1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964
     1   40   80  121  164  208  252  295  341  385  431  478  540  605
1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978
   674   745   818   894   973 1057 1142 1229 1316 1406 1496 1587 1679 1775
1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992
1871 1966 2059 2152 2245 2337 2428 2518 2607 2694 2780 2864 2944 3017
1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006
3089 3166 3239 3308 3377 3448 3521 3593 3664 3729 3793 3858 3924 3989
2007 2008
4051 4112
```

This table reads in this way: for all the observations from No. 1 to No. 39, they all belong to the year 1951. For all the observations from No. 40 to No. 79, they all belong to the year 1952. Applying the same logic, for all the observations from No. 4112 to No. 4171 (the max id number of the original dataset), they all belong to the year 2008. The result indicates that each year's number of observations is different from each other. This is the key information we rely on to split the original integrated adjacency matrix.

Now, we have collected enough information to split each year's aggregated observations from all other years.

```
# Create yearly network list to restore info later
yearly_network_coop <- list()

# Yearly adjacency matrices should be in the format of n by n, always
for (yr in 1:length(each_yr)) {
  yearly_network_coop[[yr]] <- as.matrix(coop[yr_start[yr]:yr_fin[yr],
                                          yr_start[yr]:yr_fin[yr]])
}

# Plug on the year label for each matrix inside the list
names(yearly_network_coop) <- names(each_yr)
```

```
# Check any year's matrix randomly
# yearly_network_coop[["YEAR"]]
```

Now let's take a look into the checklist of what the function `clean_blocks_by_time()` asks as inputs:

```
## Usage
clean_blocks_by_time(
  df,
  id,
  time,
  A_by_time,
  model = "bernoulli",
  out_col = "clean_block",
  seed = 1
)

## Arguments
df    panel dataframe

id    column name for unit id (string)

time    column name for time (string)

A_by_time    named list of adjacency matrices; names must match unique(df[[time]])

model    SBM model (default 'bernoulli')

out_col    name of output column to add

seed    random seed
```

`df = data`, which is the original dataset we work on. `id` in this situation means countries, because in Olar (2019), the units are nation-states. Check the names of the columns in `data`, `id = ccode`. We have conducted `time` from the `year` variable in `data`. However, it is worth noting that time should be character but not numbers.

Now here comes the key part. We conduct pre-analysis to make `yearly_network_coop`, which is essentially `A_by_time`. Note that in this paper's conceptualization and design, institutional similarity only has two outcomes—similar or dissimilar. Thus, there are only 0 and 1 inside each yearly matrix. It means we only need to choose the `bernoulli` model for SBM-based latent

homophily detection. In other papers, the adjacency matrix might contain information such as migrate flows, votes, ideological distance, etc. The other two models besides `bernoulli` is `gaussian` and `poisson`. Please make sure you choose the appropriate model to detect and recover homophily. For more information, please run this chunk:

```
install.packages("sbm")
library(sbm)
??estimateSimpleSBM
```

And then, `out_col` stores all the detected homophily based on time. For `seed`, you may pick any random seeds.

Here is the final integrated version to run `clean_blocks_by_time()`. First, pretend as if we had never done the previous pre-analysis. Assume we have examined the dataset and got to know some basic info of the original dataset, we run:

```
# Ensure time is character because the list names are "1951", "1952", ...
data2 <- data %>%
  mutate(year_chr = as.character(year))

# Function to assign dim-names year-by-year
for (yr_name in names(yearly_network_coop)) {
  ids_yr <- data2 %>%
    filter(year_chr == yr_name) %>%
    pull(ccode) %>% # The actual unit id column
    as.character()

  # Note which year it actually is as the final value left in the
  # environment, here it is year == 2008 and in yearly_network_coop,
  # for year == 2008, the matrix is 60 by 60 indeed.
  A <- yearly_network_coop[[yr_name]]

  stopifnot(nrow(A) == length(ids_yr)) # Safety check
  rownames(A) <- ids_yr
  colnames(A) <- ids_yr

  yearly_network_coop[[yr_name]] <- A
}
```

Warning: Some datasets might not contain proper dim-names. This is not a fatal problem, however, based on my previous tests, it indeed leads to some minor discrepancies. The core mechanism is that SBM is permutation-invariant in theory but not randomness-invariant in

practice. Thus, in order to ensure identical replication results, please make sure to plug on dim-names before running `clean_blocks_by_time()`. What I mean by `dim-names` is, for example, load A. You will find that the label i^{th} on the row is exactly the same as that on the column as j^{th} .

Now run the `clean_blocks_by_time()` function for this integrated matrix.

```
yearly_network_coop_noname <- lapply(yearly_network_coop, function(A) {
  A <- as.matrix(A)
  dimnames(A) <- NULL
  A
})

df_out2 <- clean_blocks_by_time(
  df = data2,
  id = "ccode",
  time = "year_chr",
  A_by_time = yearly_network_coop,
  out_col = "clean_block_coop",
  seed = 1
)
```

Check the newly added columns in `df_out2`.

```
table(df_out2$clean_block_coop)
```

The results look like this:

comm1_1951	comm1_1952	comm1_1953	comm1_1954	comm1_1955	comm1_1956	comm1_1957
17	19	19	21	21	19	16
comm1_1958	comm1_1959	comm1_1960	comm1_1961	comm1_1962	comm1_1963	comm1_1964
19	16	16	18	24	25	27
comm1_1965	comm1_1966	comm1_1967	comm1_1968	comm1_1969	comm1_1970	comm1_1971
27	26	26	27	27	29	30
comm1_1972	comm1_1973	comm1_1974	comm1_1975	comm1_1976	comm1_1977	comm1_1978
29	29	31	31	32	34	36
comm1_1979	comm1_1980	comm1_1981	comm1_1982	comm1_1983	comm1_1984	comm1_1985
41	41	41	41	41	41	42
comm1_1986	comm1_1987	comm1_1988	comm1_1989	comm1_1990	comm1_1991	comm1_1992
42	44	42	41	35	41	46
comm1_1993	comm1_1994	comm1_1995	comm1_1996	comm1_1997	comm1_1998	comm1_1999
50	47	43	47	49	50	48

comm1_2000	comm1_2001	comm1_2002	comm1_2003	comm1_2004	comm1_2005	comm1_2006
51	43	45	46	46	45	43
comm1_2007	comm1_2008	comm2_1951	comm2_1952	comm2_1953	comm2_1954	comm2_1955
44	42	11	11	11	12	14
comm2_1956	comm2_1957	comm2_1958	comm2_1959	comm2_1960	comm2_1961	comm2_1962
14	16	14	14	15	13	21
comm2_1963	comm2_1964	comm2_1965	comm2_1966	comm2_1967	comm2_1968	comm2_1969
23	24	24	23	26	25	26
comm2_1970	comm2_1971	comm2_1972	comm2_1973	comm2_1974	comm2_1975	comm2_1976
28	28	25	27	24	22	24
comm2_1977	comm2_1978	comm2_1979	comm2_1980	comm2_1981	comm2_1982	comm2_1983
24	27	28	28	29	28	28
comm2_1984	comm2_1985	comm2_1986	comm2_1987	comm2_1988	comm2_1989	comm2_1990
27	27	27	24	23	24	25
comm2_1991	comm2_1992	comm2_1993	comm2_1994	comm2_1995	comm2_1996	comm2_1997
12	11	10	11	12	12	12
comm2_1998	comm2_1999	comm2_2000	comm2_2001	comm2_2002	comm2_2003	comm2_2004
13	12	12	12	12	12	12
comm2_2005	comm2_2006	comm2_2007	comm2_2008	comm3_1951	comm3_1952	comm3_1953
12	12	12	12	7	10	11
comm3_1954	comm3_1955	comm3_1956	comm3_1957	comm3_1958	comm3_1959	comm3_1960
10	9	7	11	13	14	6
comm3_1961	comm3_1962	comm3_1963	comm3_1964	comm3_1965	comm3_1966	comm3_1967
6	7	7	10	11	12	12
comm3_1968	comm3_1969	comm3_1970	comm3_1971	comm3_1972	comm3_1973	comm3_1974
14	19	16	18	21	20	20
comm3_1975	comm3_1976	comm3_1977	comm3_1978	comm3_1979	comm3_1980	comm3_1981
21	19	20	17	13	12	13
comm3_1982	comm3_1983	comm3_1984	comm3_1985	comm3_1986	comm3_1987	comm3_1988
16	13	10	9	8	9	8
comm3_1989	comm3_1990	comm3_1991	comm3_1992	comm3_1993	comm3_1994	comm3_1995
8	11	12	9	10	15	14
comm3_1996	comm3_1997	comm3_1998	comm3_1999	comm3_2000	comm3_2001	comm3_2002
10	10	10	12	8	10	7
comm3_2003	comm3_2004	comm3_2005	comm3_2006	comm3_2007	comm3_2008	comm4_1951
7	8	8	7	5	6	4
comm4_1956	comm4_1960	comm4_1961	comm4_1962	comm4_1963	comm4_1964	comm4_1965
4	9	10	10	10	8	9
comm4_1966	comm4_1967	comm4_1968	comm4_1969	comm4_1970	comm4_1971	comm4_1972
7	7	8	12	12	11	12
comm4_1973	comm4_1974	comm4_1975	comm4_1976	comm4_1977	comm4_1978	comm4_1979
10	12	14	14	14	14	13
comm4_1980	comm4_1981	comm4_1982	comm4_1983	comm4_1984	comm4_1985	comm4_1986

	12	10	8	10	8	12	12
comm4_1987	comm4_1988	comm4_1989	comm4_1990	comm4_1991	comm4_1992	comm4_1993	
	10	8	11	9	8	6	7
comm5_1966	comm5_1967	comm5_1968	comm5_1973	comm5_1974	comm5_1975	comm5_1976	
	5	5	5	4	3	3	3
comm5_1977	comm5_1978	comm5_1984	comm5_1988				
	4	2	5	5			

And then, apply the same process to the other integrated adjacency matrix:

```
# Create yearly network list to restore info later
yearly_network_strctr <- list()

# Yearly adjacency matrices should be in the format of n by n, always
for (yr in 1:length(each_yr)) {
  yearly_network_strctr[[yr]] <- as.matrix(strctr[yr_start[yr]:yr_fin[yr],
                                           yr_start[yr]:yr_fin[yr]])
}

# Plug on the year label for each matrix inside the list
names(yearly_network_strctr) <- names(each_yr)

# Check any year's matrix randomly
# yearly_network_strctr[["YEAR"]]

# Ensure time is character because the list names are "1951", "1952", ...
data3 <- data %>%
  mutate(year_chr = as.character(year))

# Function to assign dim-names year-by-year
for (yr_name in names(yearly_network_strctr)) {
  ids_yr <- data3 %>%
    filter(year_chr == yr_name) %>%
    pull(ccode) %>%           # The actual unit id column
    as.character()

  # Note which year it actually is as the final value left in
  # the environment, here it is year == 2008 and in
  # yearly_network_strctr, for year == 2008,
  # the matrix is 60 by 60 indeed.

  A <- yearly_network_strctr[[yr_name]]
}
```

```

stopifnot(nrow(A) == length(ids_yr)) # Safety check
rownames(A) <- ids_yr
colnames(A) <- ids_yr

yearly_network_strctr[[yr_name]] <- A
}

yearly_network_strctr_noname <- lapply(yearly_network_strctr, function(A) {
  A <- as.matrix(A)
  dimnames(A) <- NULL
  A
})

df_out3 <- clean_blocks_by_time(
  df = data3,
  id = "ccode",
  time = "year_chr",
  A_by_time = yearly_network_strctr,
  out_col = "clean_block_strctr",
  seed = 1
)

```

Check the newly added columns in `df_out3`.

```
table(df_out3$clean_block_strctr)
```

Here are the results. Note that `no_block` is also one of the various values.

comm1_1960	comm1_1961	comm1_1964	comm1_1965	comm1_1966	comm1_1967
4	4	7	5	5	7
comm1_1968	comm1_1969	comm1_1970	comm1_1971	comm1_1972	comm1_1973
6	5	5	7	7	5
comm1_1974	comm1_1975	comm1_1976	comm1_1977	comm1_1978	comm1_1979
7	7	7	9	10	13
comm1_1980	comm1_1981	comm1_1982	comm1_1983	comm1_1984	comm1_1985
12	12	14	13	13	10
comm1_1986	comm1_1987	comm1_1988	comm1_1989	comm1_1990	comm1_1991
14	11	9	17	15	15
comm1_1992	comm1_1993	comm1_1994	comm1_1995	comm1_1996	comm1_1997
14	14	15	13	13	15
comm1_1998	comm1_1999	comm1_2000	comm1_2001	comm1_2002	comm1_2003

15	15	14	13	11	10
comm1_2004	comm1_2005	comm1_2006	comm1_2007	comm1_2008	comm2_1960
10	9	11	8	8	42
comm2_1961	comm2_1964	comm2_1965	comm2_1966	comm2_1967	comm2_1968
43	62	5	5	5	5
comm2_1969	comm2_1970	comm2_1971	comm2_1972	comm2_1973	comm2_1974
4	4	80	80	85	83
comm2_1975	comm2_1976	comm2_1977	comm2_1978	comm2_1979	comm2_1980
6	7	7	6	9	9
comm2_1981	comm2_1982	comm2_1983	comm2_1984	comm2_1985	comm2_1986
9	6	9	6	6	10
comm2_1987	comm2_1988	comm2_1989	comm2_1990	comm2_1991	comm2_1992
9	9	4	7	7	4
comm2_1993	comm2_1994	comm2_1995	comm2_1996	comm2_1997	comm2_1998
4	5	5	5	5	6
comm2_1999	comm2_2000	comm2_2001	comm2_2002	comm2_2003	comm2_2004
5	4	52	53	55	6
comm2_2005	comm2_2006	comm2_2007	comm2_2008	comm3_1965	comm3_1966
4	4	5	5	61	63
comm3_1967	comm3_1968	comm3_1969	comm3_1970	comm3_1975	comm3_1976
64	68	75	76	78	78
comm3_1977	comm3_1978	comm3_1979	comm3_1980	comm3_1981	comm3_1982
80	80	73	72	72	73
comm3_1983	comm3_1984	comm3_1985	comm3_1986	comm3_1987	comm3_1988
70	4	4	65	67	68
comm3_1989	comm3_1990	comm3_1991	comm3_1992	comm3_1993	comm3_1994
63	58	51	54	59	53
comm3_1995	comm3_1996	comm3_1997	comm3_1998	comm3_1999	comm3_2000
51	51	51	52	52	53
comm3_2004	comm3_2005	comm3_2006	comm3_2007	comm3_2008	comm4_1984
50	52	47	48	47	68
comm4_1985	no_block				
70	511				

Now, since we have obtained all the recovered homophily, the last step is to place them into the original structural model and re-run all the models and make a comparison, focusing on the estimands and R^2 , as well as other metrics if noticeable.

The original model usually only keeps the contagious term: $Y_{i,t+1} = \alpha_0 + \alpha_1 Y_{i,t} + \beta \frac{\sum_j (Y_{j,t} A_{ij,t})}{\sum_j A_{ij,t}} + \zeta_{i,t+1}$. The reason why β is biased is because the omitted homophily C_i simultaneously affects on contagion and the outcome $Y_{i,t}$.

Now, since we have recovered $\hat{C}_i$ as a proxy of the truly latent homophily C_i , we can plug it

back on to de-bias β , such that $Y_{i,t+1} = \alpha_0 + \alpha_1 Y_{i,t} + \beta \frac{\sum_j (Y_{j,t} A_{ij,t})}{\sum_j A_{ij,t}} + \gamma_0^T \hat{C}_{i,t} + \eta_{i,t+1}$, where $\eta_{i,t+1} = \epsilon_{i,t+1} + \gamma_2^T X_i + (\gamma_1^T C_{i,t} - \gamma_0^T \hat{C}_{i,t})$. It implies that we hope to find a $\hat{C}_i$ such that $\|\hat{C}_i - C_i\| \rightarrow 0$.

Reference

- Abbe, Emmanuel. “Community detection and stochastic block models: recent developments.” *Journal of Machine Learning Research* 18, no. 177 (2018): 1-86.
- Desmarais, Bruce A., and Skyler J. Cranmer. “Statistical inference in political networks research.” *The Oxford handbook of political networks* (2017): 203.
- Holland, Paul W., Kathryn Blackmond Laskey, and Samuel Leinhardt. “Stochastic blockmodels: First steps.” *Social networks* 5, no. 2 (1983): 109-137.
- Karrer, Brian, and Mark EJ Newman. “Stochastic blockmodels and community structure in networks.” *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 83, no. 1 (2011): 016107.
- McFowland III, Edward, and Cosma Rohilla Shalizi. “Estimating causal peer influence in homophilous social networks by inferring latent locations.” *Journal of the American Statistical Association* 118, no. 541 (2023): 707-718.
- Olar, Roman-Gabriel. “Do they know something we don’t? Diffusion of repression in authoritarian regimes.” *Journal of Peace Research* 56, no. 5 (2019): 667-681.
- Shalizi, Cosma Rohilla, and Andrew C. Thomas. “Homophily and contagion are generically confounded in observational social network studies.” *Sociological methods & research* 40, no. 2 (2011): 211-239.
- Uppala, Medha, and Bruce A. Desmarais. “Contagion, Confounding, and Causality: Confronting the Three C’s of Observational Political Networks Research.” *Political Analysis* 31, no. 3 (2023): 472-479.